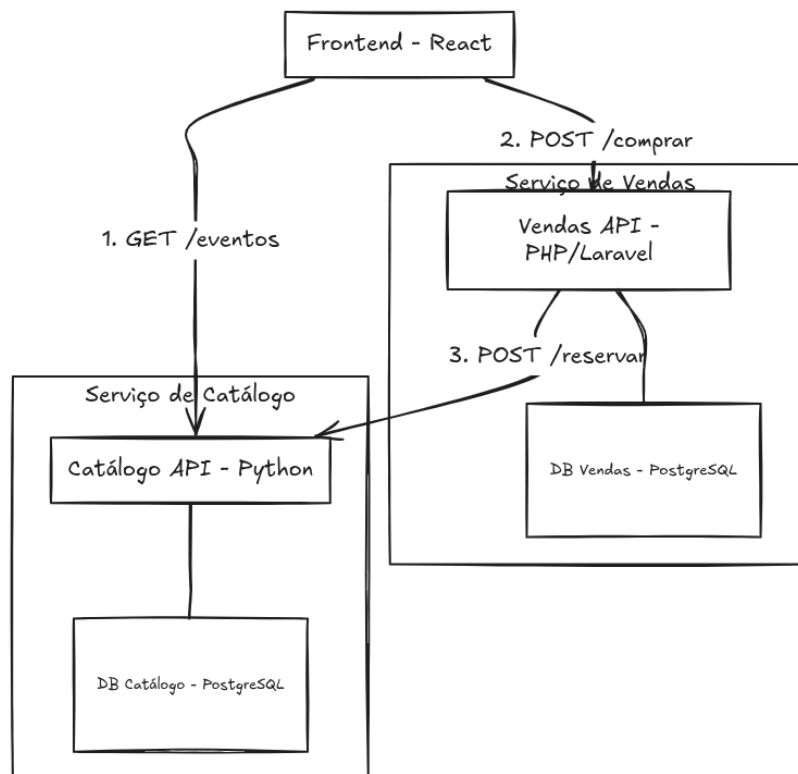


Para projetar essa arquitetura garantindo consistência (sem *overselling* de ingressos) e mantendo a simplicidade e escalabilidade, aplicaremos padrões de microsserviços como **Database-per-service** (um banco para cada serviço) e **Circuit Breaker** (para resiliência).

Parte 1 – Visão Geral

Para evitar acoplamento, cada serviço terá seu próprio banco de dados. O Frontend consumirá o catálogo diretamente para leitura (evitando sobrecarregar o serviço de vendas) e chamará o serviço de vendas apenas para a transação.

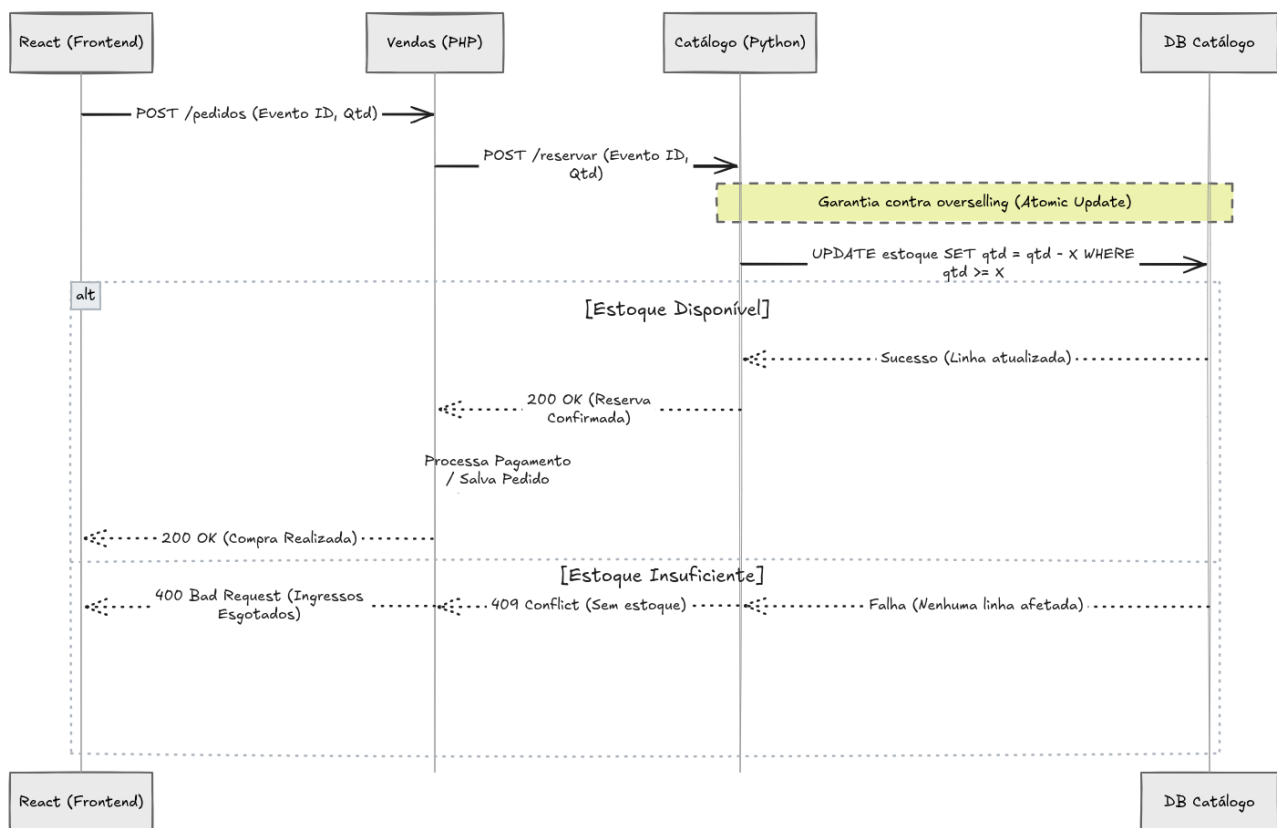


Componentes Principais:

- **Frontend (React):** Interface do usuário. Faz requisições de leitura (listar eventos) diretamente para o Catálogo e requisições de escrita (comprar) para Vendas.
- **Catálogo (Python):** Detém a fonte da verdade sobre os eventos e o estoque. Seu banco de dados precisa garantir atomicidade nas baixas de estoque.
- **Vendas (PHP/Laravel):** Orquestra o pedido. Recebe a intenção de compra, pede a reserva ao Catálogo, processa o pagamento (se houver) e consolida o pedido em seu próprio banco.

Parte 2 – Comunicação entre Serviços e Fluxo de Compra

Para garantir que não sejam vendidos ingressos além do estoque, o fluxo de **reserva de inventário** deve acontecer *antes* da confirmação do pedido.



O momento da transação é o coração desse sistema. É aqui que lidamos com o maior risco técnico em sistemas de vendas: a **condição de corrida (race condition)**, que resulta na venda de ingressos que não existem.

Como cada serviço tem seu próprio banco de dados, o Serviço de Vendas (PHP) não tem acesso direto à tabela de estoque. Ele depende totalmente do Serviço de Catálogo (Python) para saber se pode ou não prosseguir.

Aqui está o detalhamento de como essa comunicação deve ocorrer na prática.

O Problema da Concorrência (Overselling)

Imagine que resta apenas **1 ingresso** para um show. Dois usuários clicam em "Comprar" no exato mesmo milissegundo.

Se o Serviço de Catálogo (Python) for programado de forma ingênua, ele fará o seguinte:

1. `SELECT qtd FROM estoque WHERE evento_id = 1;` (Retorna 1 para o Usuário A)
2. `SELECT qtd FROM estoque WHERE evento_id = 1;` (Retorna 1 para o Usuário B)
3. Ambas as requisições veem que há estoque e enviam um UPDATE para deduzir 1.
4. Resultado: O sistema vende 2 ingressos, mas só havia 1 disponível.

A Solução: Reserva com Atualização Atômica

Para resolver isso, o Serviço de Catálogo não deve consultar o estoque antes de tentar baixar. Ele deve delegar a validação da concorrência para o próprio banco de dados (PostgreSQL) usando uma **operação atômica**.

Quando o PHP pedir ao Python para reservar o ingresso, o Python executará diretamente esta query:

SQL

```
UPDATE estoque
SET quantidade = quantidade - 1
WHERE evento_id = 1
  AND quantidade >= 1;
```

O banco de dados aplica um bloqueio em nível de linha (row-level lock) durante o UPDATE. O que acontece na prática com os mesmos dois usuários:

1. O banco processa o UPDATE do “Usuário A” primeiro. A condição `quantidade >= 1` é verdadeira. A linha é atualizada para 0. O banco retorna `1 row affected`.
2. O banco processa o UPDATE do “Usuário B” logo em seguida. A condição `quantidade >= 1` agora é falsa (pois a quantidade é 0). A linha não é modificada. O banco retorna `0 rows affected`.

O código em Python apenas verifica o número de linhas afetadas. Se for 1, a reserva foi um sucesso (retorna HTTP 200 pro PHP). Se for 0, o ingresso esgotou (retorna HTTP 409 Conflict pro PHP).

O Fluxo Completo de Comunicação (Passo a Passo)

Veja a cronologia de uma requisição bem-sucedida, misturando chamadas síncronas e assíncronas no lugar certo:

1. Fase Síncrona (A Reserva)

Nesta etapa, o usuário está esperando na tela. A comunicação precisa ser síncrona (REST) porque precisamos da resposta imediata antes de cobrar o cartão.

- **Passo 1:** O React faz um `POST /comprar` para o Serviço de Vendas (PHP).
- **Passo 2:** O PHP pausa e faz uma requisição `HTTP POST /reservar` para o Serviço de Catálogo (Python).
- **Passo 3:** O Python faz o UPDATE atômico no banco dele.
- **Passo 4:** O Python responde ao PHP: `200 OK` (Reserva garantida por, digamos, 10 minutos).

2. Fase de Orquestração (O Pagamento)

O PHP agora sabe que o ingresso está garantido.

- **Passo 5:** O PHP se comunica com o Gateway de Pagamento (Stripe, Mercado Pago, etc).
- **Passo 6:** O pagamento é aprovado.

- **Passo 7:** O PHP salva o pedido como "Pago" no seu próprio banco de dados (DB Vendas).
- **Passo 8:** O PHP responde ao React: 200 OK (Compra finalizada). O usuário vê a tela de sucesso.

3. Fase Assíncrona (O Pós-venda)

O usuário já tem a confirmação na tela. Tarefas pesadas ou demoradas não devem prender a requisição HTTP. Aqui entra a mensageria (como RabbitMQ ou Redis Pub/Sub).

- **Passo 9:** O PHP publica um evento em uma fila de mensagens: `Pedido_Confirmado { pedido_id: 123 }`.
- **Passo 10:** Processos em segundo plano (workers) escutam essa fila e executam tarefas paralelas:
 - *Worker A:* Gera o PDF do ingresso com QR Code.
 - *Worker B:* Envia o e-mail de confirmação para o usuário.
 - *Worker C:* Notifica o Serviço de Catálogo para converter a “reserva temporária” em uma “baixa definitiva”.

Essa separação garante que, se o servidor de e-mail cair, o usuário ainda consegue finalizar a compra normalmente. A mensagem de envio de e-mail ficará aguardando na fila até o serviço voltar.

Modelo	Vantagens	Desvantagens	Recomendação para este caso
Síncrono (HTTP/REST)	Simple de implementar, fluxo fácil de debugar, resposta imediata ao usuário sobre o estoque.	Maior acoplamento temporal (se o Catálogo cair, Vendas não processa), latência somada.	Ideal para a etapa de Reserva. Você precisa da resposta imediata do Catálogo para saber se pode ou não cobrar o usuário.
Assíncrono (Mensageria)	Altamente escalável, tolerante a falhas (mensagens ficam na fila se o serviço cair), baixíssimo acoplamento.	Complexidade alta (requer lidar com consistência eventual e compensações/Saga Pattern).	Ideal para o Pós-venda. Após o PHP confirmar a venda, ele emite um evento assíncrono para enviar e-mail ao usuário ou gerar o PDF do ingresso.

A escolha de uma abordagem mista (híbrida) não é um preciosismo técnico, mas sim uma decisão baseada no alinhamento direto entre os padrões de arquitetura de sistemas distribuídos e os **requisitos de negócio** de cada etapa.

Em sistemas distribuídos, sempre trocamos complexidade por escalabilidade e resiliência. A abordagem mista divide o problema separando o que exige **consistência forte** do que tolera **consistência eventual**.

Aqui está o porquê de cada paradigma ter sido aplicado no seu respectivo momento:

1. O “Caminho Crítico”: Porque a Reserva DEVE ser Síncrona

A etapa de verificar o estoque e cobrar o usuário é o que chamamos de *core domain* da transação.

Se usássemos uma abordagem **100% assíncrona** para a reserva, o fluxo seria:

1. O PHP recebe a intenção de compra, coloca na fila e responde ao usuário: “*Pedido recebido, processando...*”
2. O PHP cobra o cartão de crédito do usuário.
3. Segundos (ou minutos) depois, um *worker* lê a fila, tenta baixar o estoque no Python e descobre que os ingressos esgotaram.
4. **O problema:** Você já cobrou o usuário. Agora terá que disparar uma transação de compensação (Saga) para estornar o cartão de crédito. Isso gera custos de gateway (taxas de transação) e extrema frustração para o cliente.

O trade-off: Ao fazermos o PHP fazer uma chamada síncrona (REST) para o Python, aceitamos o **acoplamento temporal** (se o Python cair, a venda para). É o preço que pagamos para ter a garantia de que não cobraremos por algo que não temos. O objetivo aqui é evitar o estorno a todo custo.

2. O “Pós-processamento”: Porque o Pós-venda DEVE ser Assíncrono

Uma vez que o ingresso está reservado e o pagamento aprovado, o contrato de negócio com o usuário está cumprido. A transação crítica acabou.

Gerar o PDF do ingresso e enviar o e-mail via SMTP são operações de alto custo de I/O e altíssima propensão a falhas de rede.

Se usássemos uma abordagem **100% síncrona** aqui:

1. O pagamento aprova.
2. O PHP tenta gerar o PDF e enviar o e-mail na mesma requisição HTTP.
3. Se o servidor de e-mail estiver lento ou cair, a requisição do usuário fica “travada” na tela de carregamento.
4. Pior: se der *timeout* no SMTP, o PHP pode retornar um erro 500 `Internal Server Error` para o usuário, mesmo o pagamento tendo sido aprovado no gateway. O usuário achará que a compra falhou e tentará comprar de novo (gerando duplicidade).

O trade-off: Usar mensageria (como RabbitMQ ou Kafka) para o pós-venda aplica o princípio de **Isolamento de Falhas (Fault Isolation)**. O PHP apenas publica o evento `Pedido_Pago` na fila em milissegundos e libera o usuário com a tela de sucesso. Se o servidor de e-mail estiver fora do ar, a mensagem fica segura na fila. O sistema usará políticas de *retry* (como *Exponential Backoff*) e o e-mail chegará 5 minutos depois (Consistência Eventual). O negócio tolera esse atraso; o que ele não tolera é perder a venda por lentidão no SMTP.

Em resumo, a arquitetura mista permite que você tenha o melhor dos dois mundos: **exatidão transacional** onde o dinheiro está envolvido (síncrono) e **alta disponibilidade/desacoplamento** nas operações periféricas (assíncrono).

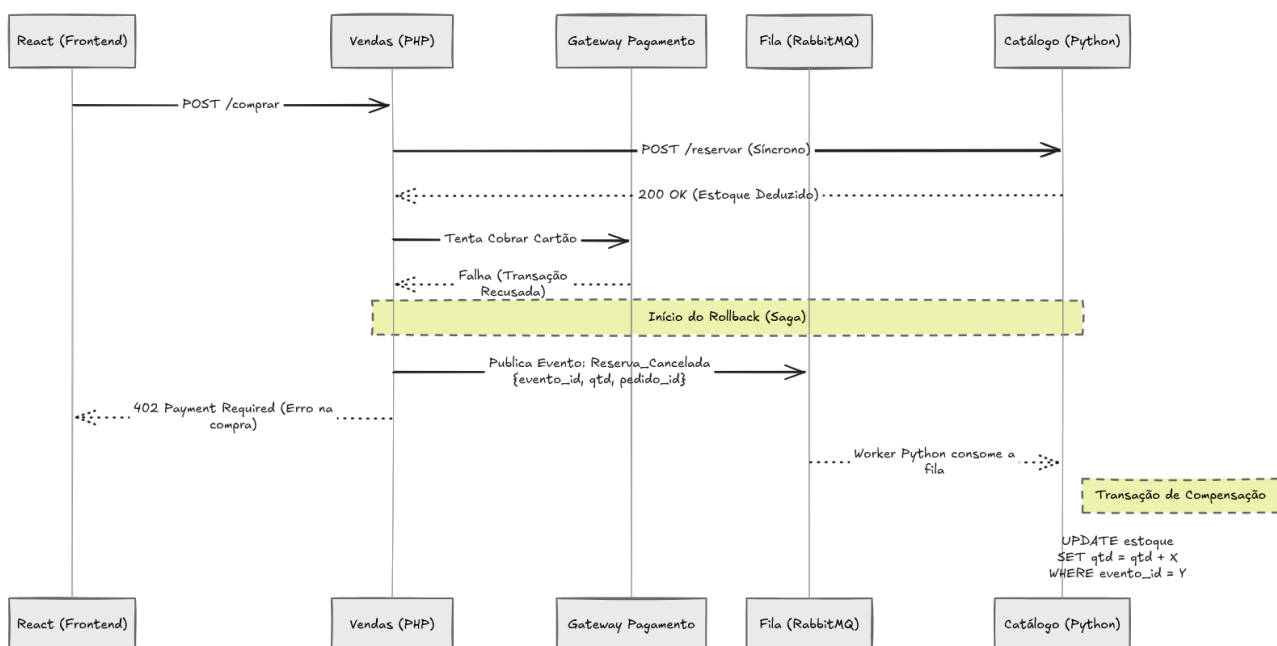
Parte 3 – Tratamento de Erros e Falhas

(Nota: Você mencionou “serviço de catálogo (PHP)”, mas pela sua premissa inicial, o Catálogo é Python e Vendas é PHP. Vou assumir o cenário onde o serviço que detém o estoque — Python — fica indisponível e o PHP precisa lidar com isso).

Se o Serviço de Catálogo (Python) estiver fora do ar ou com alta latência, a requisição de compra feita pelo Serviço de Vendas (PHP) falhará.

O que acontece com a requisição e como retornar ao usuário?

1. **Timeout e Circuit Breaker:** O serviço em PHP não deve esperar indefinidamente. Ele deve ter um *timeout* curto configurado (ex: 3 segundos) para a chamada ao Python. Para evitar gargalos, aplica-se o padrão **Circuit Breaker**: se o Python falhar seguidamente, o PHP “abre o circuito” e passa a rejeitar compras imediatamente, sem nem tentar chamar o Python, dando tempo para o serviço se recuperar.
2. **Rollback de Estado:** Como o PHP tenta reservar o ingresso *antes* de processar o pagamento, se o Python estiver indisponível, nenhuma cobrança é feita ao usuário.
3. **Resposta ao Frontend:** O serviço PHP captura a exceção de indisponibilidade (HTTP 503 ou Timeout) e traduz isso para um erro HTTP 503 (Service Unavailable) para o React.
4. **Experiência do Usuário (UX):** O React recebe o erro 503 e exibe um alerta amigável: *“Tivemos um problema temporário ao verificar a disponibilidade dos ingressos. Por favor, tente novamente em alguns instantes.”*



Para garantir que a devolução do ingresso ocorra mesmo que o Serviço de Catálogo sofra uma instabilidade logo após a falha do pagamento, usamos a mensageria (RabbitMQ) para entregar a transação de compensação.

Detalhando a Implementação do Rollback

Para que essa arquitetura seja verdadeiramente resiliente de ponta a ponta, três conceitos técnicos precisam ser implementados nos serviços:

1. Desacoplamento da Compensação Observe que o PHP não faz um `POST /cancelar-reserva` direto para o Python. Ele joga o evento no RabbitMQ. Por que? Se o pagamento falhou e, no mesmo segundo, o serviço em Python reiniciou, uma requisição HTTP falharia e o ingresso ficaria "preso" no limbo para sempre. Ao usar a fila, a mensagem de cancelamento fica segura. Quando o Python voltar à vida, ele lê a fila e devolve o ingresso ao banco.

2. Idempotência no Catálogo (Obrigatório) Em sistemas distribuídos orientados a eventos, falhas de rede podem fazer com que o RabbitMQ entregue a mesma mensagem duas vezes (*at-least-once delivery*). O serviço em Python precisa ser **idempotente**. Antes de somar +1 ao estoque, ele deve checar em uma tabela auxiliar se já processou a devolução daquele `pedido_id` específico. Se já processou, ele apenas ignora a mensagem. Isso evita que um erro de rede multiplique os ingressos disponíveis.

3. Dead Letter Queue (DLQ) O que acontece se o código Python que consome a fila tiver um bug (por exemplo, erro de sintaxe) e não conseguir processar o estorno de jeito nenhum? A mensagem não pode ficar travando a fila principal. Após X tentativas falhas (*retries*), o RabbitMQ move essa mensagem para uma fila especial chamada **Dead Letter Queue**. A equipe de engenharia pode monitorar essa DLQ para investigar o log de erro, corrigir o bug e reprocessar a mensagem manualmente, garantindo que o ingresso retorne ao estoque.

Esquema de BD para garantir idempotência e detalhamento do circuit breaker no php

Esses dois mecanismos são o que separam uma arquitetura “que funciona no caminho feliz” de um sistema distribuído maduro e pronto para produção. Como estamos lidando com consistência de dados entre serviços independentes, a engenharia precisa ser defensiva em ambas as pontas. Vamos detalhar a implementação de cada um.

1. Idempotência no Catálogo (Python + PostgreSQL)

Mensagerias como o RabbitMQ garantem entrega *at-least-once* (pelo menos uma vez). Em cenários de instabilidade de rede (ex: o Python processa a devolução do ingresso, mas a rede cai antes dele enviar o `ACK` para a fila), o RabbitMQ reenviará a mesma mensagem.

Se o Python simplesmente executar `quantidade = quantidade + 1` de forma cega, você criará ingressos fantasmas. A solução é garantir a idempotência usando o próprio banco de dados relacional como escudo transacional.

O Esquema de Banco de Dados

Criamos uma tabela dedicada para registrar as chaves de idempotência (que pode ser o ID da mensagem ou o ID do pedido cancelado).

SQL

```
CREATE TABLE eventos_processados (  
    mensagem_id VARCHAR(255) PRIMARY KEY, -- UUID gerado pelo serviço de Vendas  
(PHP)   
    pedido_id INT NOT NULL,  
    processado_em TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
) ;
```

A Lógica em Python (Transação Atômica)

A chave técnica aqui é usar o recurso ON CONFLICT (Upsert) do PostgreSQL amarrado na mesma transação (ACID) que atualiza o estoque.

Se a inserção na tabela eventos_processados falhar por chave duplicada, sabemos que é um reenvio e abortamos a atualização do estoque em segurança.

Python

```
# Exemplo conceitual usando psycopg2 ou SQLAlchemy raw  
def processar_compensacao_reserva(mensagem):  
    # mensagem = {"mensagem_id": "uuid-123", "pedido_id": 456, "evento_id": 10,  
    "qtd": 2}
```

```
    try:  
        # Inicia a transação  
        db.execute("BEGIN;")
```

```
        # Tenta registrar o evento. Se já existir, não faz nada (DO NOTHING) e  
        # não retorna ID.  
        result = db.execute("""  
            INSERT INTO eventos_processados (mensagem_id, pedido_id)  
            VALUES (%(mensagem_id)s, %(pedido_id)s)  
            ON CONFLICT (mensagem_id) DO NOTHING  
            RETURNING mensagem_id;  
        """, mensagem)
```

```
        if not result.fetchone():  
            # A mensagem já foi processada anteriormente.  
            # Fazemos o rollback (por segurança) e retornamos sucesso para a  
            # fila.  
            db.execute("ROLLBACK;")  
            return "ACK" # Informa ao RabbitMQ que pode descartar a mensagem
```

```
        # Se chegou aqui, é a primeira vez que vemos esta mensagem.  
        # Executamos a devolução do estoque.  
        db.execute("""  
            UPDATE estoque  
            SET quantidade = quantidade + %(qtd)s  
            WHERE evento_id = %(evento_id)s;  
        """, mensagem)
```

```
        # Comita ambas as operações (registro do evento e update do estoque)  
        # atômica.  
        db.execute("COMMIT;")  
        return "ACK"
```

```
    except Exception as e:  
        db.execute("ROLLBACK;")
```



```
# Em caso de erro de banco, retornamos NACK para o RabbitMQ tentar
novamente mais tarde
return "NACK"
```

2. Circuit Breaker no Vendas (PHP / Laravel)

O Serviço de Vendas faz chamadas síncronas HTTP para o Catálogo. Se o Python começar a demorar 10 segundos para responder, e o Vendas receber 500 requisições por segundo, as threads do servidor PHP (FPM ou Swoole) vão se esgotar rapidamente. O sistema inteiro cai (Cascading Failure).

O **Circuit Breaker** evita isso agindo como um disjuntor elétrico. Ele tem três estados:

- **Closed (Fechado):** Fluxo normal. As requisições passam.
- **Open (Aberto):** Disjuntor desarmado. Falhas excederam o limite. O PHP lança exceção imediatamente, sem nem tentar fazer a requisição HTTP.
- **Half-Open (Meio Aberto):** Após um tempo (ex: 30s), ele deixa passar uma requisição para testar. Se der certo, ele fecha (normaliza). Se falhar, abre de novo.

Implementação Nativa no Laravel usando Cache

Embora existam pacotes como `ackintosh/ganesha`, você pode implementar um Circuit Breaker eficiente usando o Cache do Laravel (preferencialmente suportado por Redis, para ser compartilhado entre todas as instâncias do PHP).

PHP

```
use Illuminate\Support\Facades\Http;
use Illuminate\Support\Facades\Cache;

class CatalogServiceClient
{
    private string $circuitKey = 'circuit_breaker:catalog';
    private string $failuresKey = 'circuit_breaker:catalog_failures';

    private int $maxFailures = 5; // Quantas falhas antes de abrir o
    // circuito
    private int $timeoutSeconds = 30; // Tempo que o circuito fica "Aberto"
    // antes de testar (Half-Open)

    public function reservarIngresso(int $eventoId, int $qtd)
    {
        // 1. Verifica se o circuito está ABERTO
        if (Cache::has($this->circuitKey)) {
            // Fail-fast: Rejeita imediatamente sem sobrecarregar a rede ou o
            // Python
            throw new \Exception("Serviço de catálogo temporariamente
            indisponível (Circuit Open).");
        }

        try {
            // 2. Tenta fazer a requisição (Timeout restrito é fundamental!)
            $response = Http::timeout(2)->post("http://catalog-api/reservar", [
                'evento_id' => $eventoId,
                'qtd' => $qtd
            ]);

            if ($response->successful()) {
                // Sucesso: Reseta o contador de falhas (Estado Closed mantido)
                Cache::forget($this->failuresKey);
            }
        }
    }
}
```

```

        return $response->json();
    }

    // Tratamento para HTTP 4xx (Erros de negócio, ex: sem estoque). Não
    // aciona o disjuntor.
    if ($response->clientError()) {
        throw new \DomainException("Estoque insuficiente.");
    }

    // HTTP 5xx: Falha no serviço
    $this->recordFailure();

    } catch (\Illuminate\Http\Client\ConnectionException $e) {
        // Timeout de rede ou DNS resolution failed
        $this->recordFailure();
    }
}

private function recordFailure()
{
    // Incrementa o número de falhas
    $failures = Cache::increment($this->failuresKey);

    // Se atingiu o limite, ABRE o circuito
    if ($failures >= $this->maxFailures) {
        // O circuito ficará aberto por $timeoutSeconds
        Cache::put($this->circuitKey, true, $this->timeoutSeconds);
        Cache::forget($this->failuresKey); // Prepara para o estado Half-
        // Open no futuro
    }

    throw new \Exception("Falha de comunicação com o Catálogo.");
}
}

```

Vantagens dessa abordagem:

1. **Fail-Fast:** No momento em que o limite de `maxFailures` é atingido, as próximas requisições do Frontend receberão o erro quase instantaneamente (leitura no Redis em milissegundos), poupando os workers do PHP.
2. **Auto-Recuperação (Half-Open embutido):** Como o `$circuitKey` tem um TTL (Tempo de vida) no Cache de 30 segundos, após esse período a chave expira. A próxima requisição tentará acessar o Python novamente. Se falhar, registra a falha e reabre rapidamente. Se der sucesso, o fluxo normal é retomado.

Implementação:

Parte 1 – Backend (Integração PHP -> Python)

Escrever uma função em PHP de uma classe `VendaController`. Ela recebe o ID do evento e a quantidade. Ela precisa chamar a API do Catálogo (Python) via HTTP POST para reservar o ingresso. Se der certo, salva a venda no banco local.

Parte 2 – Frontend (React consumindo API)

Escreva um componente simples em React que tem um botão 'Comprar'. Ao clicar, ele chama a API. Trate o estado de 'Carregando' e o 'Erro' caso a API retorne à falha.

Parte 1 – Backend (Integração PHP -> Python)

Nesta classe `VendaController` em Laravel, utilizamos a *Facade* `Http` para realizar a chamada síncrona ao microserviço em Python. É fundamental definir um limite de tempo (`timeout`) para evitar que o servidor PHP congele caso o Python demore a responder.

PHP

```
<?php

namespace App\Http\Controllers;

use Illuminate\Http\Request;
use Illuminate\Support\Facades\Http;
use App\Models\Venda;
use Exception;

class VendaController extends Controller
{
    public function store(Request $request)
    {
        // Validação básica da entrada
        $request->validate([
            'evento_id' => 'required|integer',
            'quantidade' => 'required|integer|min:1',
        ]);

        $eventoId = $request->input('evento_id');
        $quantidade = $request->input('quantidade');

        try {
            // 1. Chama a API do Catálogo (Python) via POST
            // O timeout(3) garante que o PHP não espere mais que 3 segundos
            $response = Http::timeout(3)->post(env('CATALOG_API_URL') .
'/reservar', [
                'evento_id' => $eventoId,
                'quantidade' => $quantidade
            ]);

            // 2. Verifica se houve conflito (ex: Ingressos Esgotados)
            if ($response->status() === 409) {
                return response()->json(['message' => 'Ingressos esgotados.'],
400);
            }

            // Verifica qualquer outro erro no Catálogo (500, 404, etc)
            if (!$response->successful()) {
                throw new Exception('Falha ao reservar ingresso no catálogo.');
```

```

Vendas) // 3. Reserva confirmada! Salva a venda no banco local (PostgreSQL

    $venda = Venda::create([
        'evento_id' => $eventoId,
        'quantidade' => $quantidade,
        'status' => 'confirmada'
    ]);

    return response()->json([
        'message' => 'Compra realizada com sucesso!',
        'venda_id' => $venda->id
    ], 200);

} catch (Exception $e) {
    // Captura falhas de comunicação (Timeouts, serviço fora do ar)
    // Log::error($e->getMessage()); // Idealmente, registrar no log

    return response()->json([
        'message' => 'Sistema de reservas indisponível no momento. Tente
novamente em instantes.'
    ], 503);
}
}
}

```

Parte 2 – Frontend (React consumindo a API PHP)

No React, precisamos gerenciar três estados fundamentais durante uma transação financeira ou de estoque: carregamento (para desabilitar múltiplos cliques), erro (feedback ao usuário) e sucesso.

JavaScript

```

import React, { useState } from 'react';

export default function BotaoComprar({ eventoId }) {
    // Estados do componente
    const [isLoading, setIsLoading] = useState(false);
    const [error, setError] = useState(null);
    const [success, setSuccess] = useState(false);

    const handleComprar = async () => {
        setIsLoading(true);
        setError(null); // Reseta erros de tentativas anteriores
        setSuccess(false);

        try {
            const response = await fetch('http://api-vendas.local/api/vendas', {
                method: 'POST',
                headers: {
                    'Content-Type': 'application/json',
                    'Accept': 'application/json'
                },
                body: JSON.stringify({
                    evento_id: eventoId,
                    quantidade: 1 // Fixo em 1 para o exemplo, poderia vir de um
<select>
                })
            });

            const data = await response.json();

            // Se a API retornar qualquer status de erro (400, 503, etc)

```

```

        if (!response.ok) {
            throw new Error(data.message || 'Ocorreu um erro ao processar
sua compra.');
```

```

        }

        // Se passou, a compra foi finalizada
        setSuccess(true);

    } catch (err) {
        setError(err.message);
    } finally {
        // Sempre remove o estado de carregamento ao final, dando erro ou
sucesso
        setIsLoading(false);
    }
};

return (
    <div className="compra-widget">
        {success ? (
            <div className="alert alert-success">
                <strong>Parabéns!</strong> Sua compra foi realizada com
sucesso.
            </div>
        ) : (
            <>
                <button
                    onClick={handleComprar}
                    disabled={isLoading}
                    className="btn-comprar"
                >
                    {isLoading ? 'Processando Reserva...' : 'Comprar
Ingresso'}
                </button>

                {error && (
                    <div style={{ color: 'red', marginTop: '10px', fontSize:
'14px' }}>
                        {error}
                    </div>
                )}
            </>
        )}
    </div>
);
}

```

1. Documentação de APIs (Integração Backend-Frontend)

Para garantir que o time de frontend tenha autonomia e contratos claros, a documentação não deve ser um documento estático (como um PDF ou Wiki manual), mas sim uma documentação viva e interativa baseada no padrão **OpenAPI (Swagger)**.

- **Implementação:** No microserviço Python (especialmente se usar FastAPI), o Swagger já é gerado nativamente. No PHP/Laravel, utilizaria pacotes como L5-Swagger ou Scribe para gerar a documentação a partir de anotações no código ou rotas.
- **O que deve constar:**

- **Schemas (DTOs):** Tipos de dados exatos esperados (ex: `evento_id` é inteiro, `quantidade` é inteiro > 0).
- **Respostas Possíveis:** O frontend precisa saber não apenas o caminho feliz (200 OK), mas como tratar as falhas. A documentação deve mapear explicitamente erros como 400 Bad Request (validação), 409 Conflict (sem estoque) e 503 Service Unavailable (Circuit breaker aberto).
- **Autenticação:** Como enviar os headers (ex: `Authorization: Bearer <token>`).
- **Mocking:** Fornecer um servidor de Mock (usando ferramentas como Postman ou Stoplight) para que o frontend possa desenvolver a interface mesmo que a API ainda esteja sendo construída.

2. Segurança e Segregação de Dados

O risco de esgotarem o estoque sem pagar (ataque de negação de inventário) é real em sistemas de ingressos.

Proteção contra falsas reservas:

1. **Rate Limiting / Throttling:** Limitar quantas requisições de reserva um IP ou usuário logado pode fazer por minuto (facilmente configurável via middleware no Laravel).
2. **TTL (Time-To-Live) da Reserva:** O microsserviço Python deve registrar um *timestamp* de expiração para cada reserva (ex: 15 minutos). Se o pagamento não for confirmado pelo serviço de Vendas dentro desse tempo, um processo em background (Worker/Cron) devolve os ingressos ao estoque automaticamente.
3. **Proteção contra Bots:** Implementar na camada pública (API Gateway ou no próprio Laravel) validações como reCAPTCHA ou Cloudflare Turnstile no momento do checkout.

Segregação nas Camadas:

- **Comunicação Interna Segura:** O Frontend **nunca** fala diretamente com o Python para comprar. Ele fala com o PHP. A rota `POST /reservar` do Python não deve estar exposta à internet. A comunicação entre PHP e Python deve ocorrer em uma rede privada (VPC) e/ou protegida por mTLS ou API Keys internas.
- **Isolamento de PII (Personally Identifiable Information):** O banco de dados do Catálogo (Python) não precisa e não deve saber quem é o comprador. Ele lida apenas com `evento_id`, `quantidade` e um `pedido_id` anonimizado. Toda a informação sensível do usuário (nome, CPF, cartão) fica restrita ao banco do Serviço de Vendas (PHP), facilitando a conformidade com LGPD.

3. Escalabilidade (Alta Concorrência)

Em um cenário de pico extremo (ex: abertura de vendas para um show muito aguardado), o **Serviço de Catálogo (Python)** e o **seu Banco de Dados (PostgreSQL)** serão os mais impactados, pois concentram o gargalo natural do sistema: o controle de concorrência na mesma linha de estoque.

Sugestões de Solução:

1. **Cache de Leitura agressivo (Redis):** As chamadas de GET /eventos não devem bater no banco relacional durante o pico. O Python deve servir esses dados a partir do Redis. Apenas a intenção de compra chega ao PostgreSQL.
2. **Fila de Espera Virtual (Virtual Waiting Room):** Para evitar que milhares de requisições de compra tentem fazer o UPDATE no banco no exato mesmo segundo (o que esgotaria as conexões e derrubaria o banco), implementa-se uma fila na borda da aplicação (via Cloudflare ou Redis). Os usuários recebem um "lugar na fila" e são liberados em lotes suportáveis pelo banco de dados (ex: 500 por segundo).
3. **PgBouncer (Connection Pooling):** Essencial para o PostgreSQL, garantindo que o alto número de *threads* das aplicações (PHP e Python) não ultrapasse o limite de conexões físicas que o banco de dados consegue gerenciar.

4. Extra: Lidando com um Gargalo de Performance no Time

Ao identificar um gargalo causado por um colega ou pelo próprio time, a abordagem deve ser baseada em **dados (evidências)** e não em opiniões, promovendo um ambiente colaborativo.

Como eu faria:

1. **Medir primeiro (Profiling):** Eu não chegaria dizendo "seu código está lento". Eu utilizaria ferramentas de APM (como New Relic, Datadog) ou *profilers* (como Telescope no Laravel ou cProfile no Python) para extrair métricas claras. Ex: *"A função X está demorando 1.2 segundos por causa do problema de N+1 queries"*.
2. **Prova de Conceito (PoC):** Escreveria a melhoria localmente (ex: adicionando *Eager Loading* no Eloquent do Laravel ou criando um índice no banco de dados) e rodaria testes de carga básicos para comparar.
3. **Revisão de Código (Pull Request):** Abriria um PR com a refatoração e colocaria na descrição o "Antes" e o "Depois", demonstrando tecnicamente o ganho de performance (ex: de 1.2s para 80ms).
4. **Feedback Positivo:** Chamaria os desenvolvedores envolvidos para apresentar a descoberta de forma construtiva, usando isso como uma oportunidade de nivelamento técnico para a equipe inteira (ex: "Descobri uma forma de otimizar nossas consultas aqui que podemos aplicar em outros *endpoints* também").

Como posso implementar tecnicamente o padrão de Fila de Espera Virtual (Virtual Waiting Room) usando Redis para proteger o banco de dados durante os picos de acessos?

Para implementar uma Fila de Espera Virtual (Virtual Waiting Room) que atue como um buffer e proteja o banco de dados relacional, a estrutura de dados ideal no Redis é o **Sorted Set (ZSET)**. Ele garante ordenação FIFO (First In, First Out) com complexidade de tempo $O(\log(N))$ para inserções e consultas de posição, o que é altamente eficiente mesmo com milhões de usuários.

A arquitetura se divide em duas áreas de memória no Redis: a fila de espera (ZSET) e a lista de sessões ativas (SET ou HASH com TTL). O fluxo contorna o processamento pesado no banco de dados isolando o tráfego na camada de cache.

1. Estruturas no Redis

- **fila_espera:{evento_id} (ZSET):** Armazena os usuários aguardando. O score é o timestamp em milissegundos (garantindo a ordem de chegada). O member é um UUID da sessão do usuário.
- **sessoes_liberadas:{evento_id} (SET):** Armazena os UUIDs dos usuários que ganharam o direito de comprar. Possui um TTL (Time-To-Live) para que a vaga expire se o usuário abandonar o fluxo.

2. O Fluxo de Entrada (Ingress)

Quando o usuário acessa a página do evento, o frontend faz uma requisição para um endpoint leve no Laravel.

1. O PHP gera um UUID único para o cliente.
2. Adiciona o UUID ao ZSET usando o timestamp atual.
3. Retorna o UUID e a posição atual.

PHP

```
// Laravel - Redis Facade
$uuid = Str::uuid()->toString();
$timestamp = microtime(true);
$eventoId = 123;

Redis::zadd("fila_espera:{$eventoId}", $timestamp, $uuid);
```

3. Consulta de Posição (Polling)

O frontend entra em um laço de polling (ex: a cada 10 segundos) consultando o status do UUID. O Laravel verifica primeiro se o usuário já foi liberado; caso contrário, calcula a posição na fila.

PHP

```
$isLiberado = Redis::sismember("sessoes_liberadas:{$eventoId}", $uuid);

if ($isLiberado) {
    return response()->json(['status' => 'liberado', 'token compra' => $uuid]);
}

// ZRANK retorna o índice base 0. Somamos 1 para a posição real.
$posicao = Redis::zrank("fila_espera:{$eventoId}", $uuid);

if ($posicao !== false) {
    return response()->json(['status' => 'aguardando', 'posicao' => $posicao + 1]);
}
```


4. O Gatekeeper (Liberação de Lotes)

Este é o motor da fila. Um processo em background (Worker no Laravel ou um daemon em Python) roda continuamente (ex: a cada 5 segundos) avaliando a saúde do banco de dados (PostgreSQL) e liberando um número fixo de usuários.

Para evitar condições de corrida entre múltiplos workers, esta operação deve ser atômica usando um **Lua Script** no Redis ou comandos como ZPOPMIN (disponível no Redis 5.0+).

PHP

```
// Worker liberando 500 usuários por vez
$ vagas = 500;
$eventoId = 123;

// Remove e retorna os 500 usuários mais antigos (menor timestamp)
$usuariosLiberados = Redis::zpopmin("fila_espera:{ $eventoId}", $vagas);

foreach ($usuariosLiberados as $uuid => $score) {
    // Adiciona ao SET de sessões liberadas
    Redis::sadd("sessoes_liberadas:{ $eventoId}", $uuid);
}

// Define um TTL global para a chave ou usa chaves individuais para expiração
// exata
// Se um usuário não concluir a compra em 10 minutos, ele perde a vaga.
```

5. Middleware de Proteção no Checkout

O endpoint real de compra (POST /comprar), que se comunica com o Serviço de Catálogo, agora fica protegido por um Middleware. Nenhum request chega ao controlador (e consequentemente ao PostgreSQL) sem que o UUID esteja no set de sessões liberadas.

PHP

```
// Middleware de Validação de Fila
public function handle(Request $request, Closure $next)
{
    $uuid = $request->header('X-Fila-Token');
    $eventoId = $request->input('evento_id');

    if (!$uuid || !Redis::sismember("sessoes_liberadas:{ $eventoId}", $uuid)) {
        return response()->json(['error' => 'Acesso negado. Retorne à fila.'],
403);
    }

    $response = $next($request);

    // Após a conclusão bem-sucedida da compra, removemos a permissão
    // para evitar reuso do token.
    if ($response->status() === 200) {
        Redis::srem("sessoes_liberadas:{ $eventoId}", $uuid);
    }

    return $response;
}
```

Essa arquitetura reduz a carga no banco de dados de $O(N)$ requisições simultâneas (onde N é o total de acessos) para uma constante rigorosamente controlada pelo Worker, convertendo um pico de tráfego destrutivo em um fluxo de processamento linear.